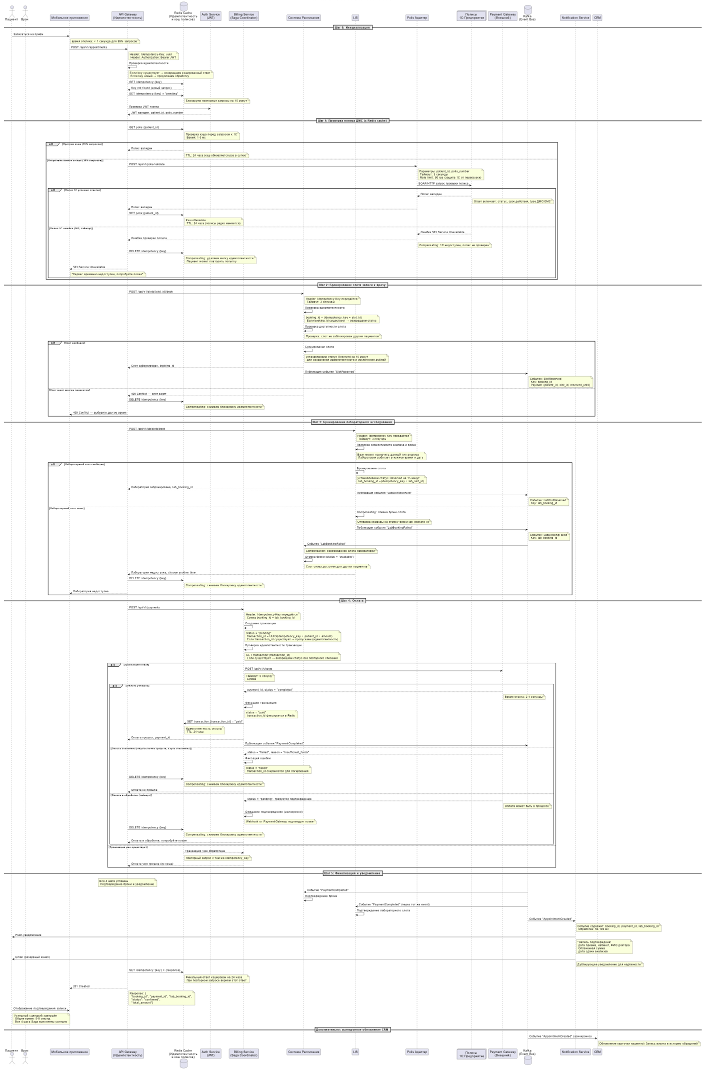
Успешный сценарий: Запись на приём + Лабораторное исследование
(Choreography-based Saga с учётом интеграции с 1С)


Пациент
Врач
Мобильное приложение
API Gateway (Идемпотентность)
Redis Cache (Идемпотентность и кэш полисов)
Auth Service (JWT)
Billing Service (Saga Coordinator)
Система Расписания
LIS
Polis Адаптер
Полисы 1С:Предприятие
Payment Gateway (Внешний)
Kafka (Event Bus)
Notification Service
CRM
Шаг 0: Инициализация
Записаться на приём
время отклика: < 1 секунда для 99% запросов
POST /api/v1/appointments
Header: Idempotency-Key: uuid
Header: Authorization: Bearer JWT
Проверка идемпотентности
Если key существует → возвращаем кэшированный ответ
Если key новый → продолжаем обработку
GET idempotency:{key}
Key not found (новый запрос)
SET idempotency:{key} = "pending"
Блокируем повторные запросы на 15 минут
Проверка JWT токена
JWT валиден, patient_id, polis_number
Шаг 1: Проверка полиса ДМС (с Redis cache)
GET polis:{patient_id}
Проверка кэша перед запросом к 1С
Время: 1-3 мс
alt
[Прогрев кэша (70% запросов)]
Полис валиден
TTL: 24 часа (кэш обновляется раз в сутки)
[Отсутствие записи в кэше (30% запросов)]
POST /api/v1/polis/validate
Параметры: patient_id, polis_number
Таймаут: 3 секунды
Rate limit: 50 rps (защита 1С от перегрузки)
SOAP/HTTP запрос проверки полиса
alt
[Полис 1С успешно ответил]
Полис валиден
Ответ включает: статус, срок действия, type ДМС/ОМС
Полис валиден
SET polis:{patient_id}
Кэш обновлён
TTL: 24 часа (полисы редко меняются)
[Полис 1С ошибка (503, таймаут)]
Ошибка 503 Service Unavailable
Ошибка проверки полиса
Compensating: 1С недоступен, полис не проверен
DELETE idempotency:{key}
Compensating: удаляем метку идемпотентности
Пациент может повторить попытку
503 Service Unavailable
"Сервис временно недоступен, попробуйте позже"
Шаг 2: Бронирование слота записи к врачу
POST /api/v1/slots/{slot_id}/book
Header: Idempotency-Key передаётся
Таймаут: 3 секунды
Проверка идемпотентности
booking_id = (idempotency_key + slot_id)
Если booking_id существует → возвращаем статус
Проверка доступности слота
Проверка: слот не заблокирован другим пациентом
alt
[Слот свободен]
Бронирование слота
устанавливаем статус Reserved на 15 минут
для сохранения идемпотентности и исключения дублей
Слот забронирован, booking_id
Публикация события "SlotReserved"
Событие: SlotReserved
Key: booking_id
Payload: {patient_id, slot_id, reserved_until}
[Слот занят другим пациентом]
409 Conflict — слот занят
DELETE idempotency:{key}
Compensating: снимаем блокировку идемпотентности
409 Conflict — выберите другое время
Шаг 3: Бронирование лабораторного исследования
POST /api/v1/lab/slots/book
Header: Idempotency-Key передаётся
Таймаут: 3 секунды
Проверка совместимости анализа и врача
Врач может назначить данный тип анализа
Лаборатория работает в нужное время и дату
alt
[Лабораторный слот свободен]
Бронирование слота
устанавливаем статус Reserved на 15 минут
lab_booking_id =(idempotency_key + lab_slot_id)
Лаборатория забронирована, lab_booking_id
Публикация события "LabSlotReserved"
Событие: LabSlotReserved
Key: lab_booking_id
[Лабораторный слот занят]
Compensating: отмена брони слота
Отправка команды на отмену брони lab_booking_id
Публикация события "LabBookingFailed"
Событие: LabBookingFailed
Key: lab_booking_id
Событие "LabBookingFailed"
Compensation: освобождение слота лаборатории
Отмена брони (status = "available")
Слот снова доступен для других пациентов
Лаборатория недоступна, choose another time
DELETE idempotency:{key}
Compensating: снимаем блокировку идемпотентности
Лаборатория недоступна
Шаг 4: Оплата
POST /api/v1/payments
Header: Idempotency-Key передаётся
Сумма booking_id + lab_booking_id
Создание транзакции
status = "pending"
transaction_id = UUID(idempotency_key + patient_id + amount)
Если transaction_id существует → пропускаем (идемпотентность)
Проверка идемпотентности транзакции
GET transaction:{transaction_id}
Если существует → возвращаем статус без повторного списания
alt
[Транзакция новая]
POST /api/v1/charge
Таймаут: 5 секунд
Сумма
alt
[Оплата успешна]
payment_id, status = "completed"
Время ответа: 2-4 секунды
Фиксация транзакции
status = "paid"
transaction_id фиксируется в Redis
SET transaction:{transaction_id} = "paid"
Идемпотентность оплаты
TTL: 24 часа
Оплата прошла, payment_id
Публикация события "PaymentCompleted"
[Оплата отклонена (недостаточно средств, карта отклонена)]
status = "failed", reason = "insufficient_funds"
Фиксация ошибки
status = "failed"
transaction_id сохраняется для логирования
DELETE idempotency:{key}
Compensating: снимаем блокировку идемпотентности
Оплата не прошла
[Оплата в обработке (таймаут)]
status = "pending", требуется подтверждение
Оплата может быть в процессе
Ожидание подтверждения (асинхронно)
Webhook от PaymentGateway подтвердит позже
DELETE idempotency:{key}
Compensating: снимаем блокировку идемпотентности
Оплата в обработке, попробуйте позже
[Транзакция уже существует]
Транзакция уже обработана
Повторный запрос с тем же idempotency_key
Оплата уже прошла (из кэша)
Шаг 5: Финализация и уведомление
Все 4 шага успешны
Подтверждение брони и уведомление
Событие "PaymentCompleted"
Подтверждение брони
Событие "PaymentCompleted" (через тот же event)
Подтверждение лабораторного слота
Событие "AppointmentCreated"
Событие содержит: booking_id, payment_id, lab_booking_id
Обработка: 50-100 мс
Push-уведомление
"Запись подтверждена!
дата приема, кабинет, ФИО доктора
Оплаченная сумма
дата сдачи анализов
Email (резервный канал)
Дублирующее уведомление для надёжности
SET idempotency:{key} = {response}
Финальный ответ кэширован на 24 часа
При повторном запросе вернём этот ответ
201 Created
Response: {
"booking_id", "payment_id", "lab_booking_id",
"status": "confirmed",
"total_amount"}
Отображение подтверждения записи
Успешный сценарий завершён
Общее время: 5-8 секунд
Все 4 шага Saga выполнены успешно
Дополнительно: асинхронное обновление CRM
Событие "AppointmentCreated" (асинхронно)
Обновление карточки пациента\ Запись визита в историю обращений